QUBIT ZERO TO HERO

Vectors, matrices and operators

S02 · Mathematical foundations
LaTeX derivations with NumPy and QuTiP checks

Column vectors and components

$$v = \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} \in \mathbb{C}^{2\times 1}, \qquad v_1, v_2 \in \mathbb{C}$$

A column vector stores ordered complex components.

Its shape records two rows and one column.

A ket is a quantum-state column vector with a specified basis.

Matrix shapes and multiplication

$$A \in \mathbb{C}^{m \times n}, \quad v \in \mathbb{C}^{n \times 1}, \quad Av \in \mathbb{C}^{m \times 1}$$

$$(Av)_i = \sum_{j=1}^{n} A_{ij} v_j$$

The shared dimension n must agree.

Each output component is a row of A multiplied by the column v.

Example 1 · Normalizing a vector

$$v = \begin{pmatrix} 1 \\ i \end{pmatrix}, \qquad \|v\| = \sqrt{|1|^2 + |i|^2} = \sqrt{2}$$

$$u = \frac{v}{\|v\|} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \end{pmatrix}, \qquad \|u\| = 1$$

Normalization divides every component by the same positive norm.

The zero vector cannot be normalized.

Transpose, conjugate and adjoint

$$v = \begin{pmatrix} 1 \\ i \end{pmatrix}, \quad v^{\mathsf{T}} = \begin{pmatrix} 1 & i \end{pmatrix}, \quad v^{\dagger} = \begin{pmatrix} 1 & -i \end{pmatrix}$$

$$A^{\dagger} = (A^{*})^{\mathsf{T}}, \qquad (AB)^{\dagger} = B^{\dagger} A^{\dagger}$$

Transpose exchanges rows and columns.

Adjoint conjugates every entry and then transposes.

The complex inner product

$$\langle u, v \rangle = u^\dagger v = \sum_{j=1}^{n} u_j^* v_j$$

$$\langle v, u \rangle = \langle u, v \rangle^*$$

The first vector is conjugated in this convention.

The inner product is a complex scalar.

Orthogonal vectors have inner product zero.

Norm and overlap bound

$$\|v\|^2 = \langle v, v \rangle = \sum_j |v_j|^2 \geq 0$$

$$|\langle u, v \rangle| \leq \|u\| \, \|v\|$$

The Euclidean norm is the nonnegative square root.

For normalized vectors, the squared overlap lies between zero and one.

Example 2 · A complex orthogonality check

$$u = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 \\ i \end{pmatrix}, \quad v = \frac{1}{\sqrt{2}}\begin{pmatrix} i \\ 1 \end{pmatrix}$$

$$u^{\dagger} v = \frac{1}{2}\big(1 \cdot i + (-i) \cdot 1\big) = 0$$

Both vectors have norm one, and their inner product vanishes.

Without conjugation, the same component sum would give i.

Outer products are operators

$$u \in \mathbb{C}^{n\times 1}, \quad v^\dagger \in \mathbb{C}^{1\times n}$$

$$uv^\dagger \in \mathbb{C}^{n\times n}, \qquad (uv^\dagger)_{ij} = u_i v_j^*$$

An inner product gives a scalar. An outer product gives a matrix.

A normalized vector defines a projector onto its direction.

Example 3 · A rank-one projector

$$P = uu^\dagger = \frac{1}{2}\begin{pmatrix} 1 & -i \\ i & 1 \end{pmatrix}$$

$$P^\dagger = P, \quad P^2 = P, \quad Pu = u, \quad Pv = 0$$

The projector keeps u and removes its orthogonal direction v.

Its trace is one, matching its one-dimensional range.

Hermitian operators

$$A^\dagger = A$$

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \qquad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$$

A Hermitian matrix equals its own adjoint.

Its eigenvalues are real, and it has an orthonormal eigenbasis.

Why Hermitian expectations are real

$$a = v^\dagger A v$$

$$a^* = (v^\dagger A v)^\dagger = v^\dagger A^\dagger v = v^\dagger A v = a$$

A scalar equal to its own conjugate is real.

For a normalized state, this is the expectation of the observable A.

Unitary operators preserve inner products

$$U^\dagger U = I, \qquad U^{-1} = U^\dagger$$

$$\langle Uu, Uv \rangle = u^\dagger U^\dagger U v = u^\dagger v$$

A square unitary matrix preserves lengths and overlaps.

Its columns form an orthonormal basis.

Hermitian and unitary are different tests

$$Z^\dagger = Z, \quad Z^\dagger Z = I$$

$$(2Z)^\dagger = 2Z, \quad (2Z)^\dagger (2Z) = 4I$$

$$U = \mathrm{diag}(e^{-i\theta}, e^{i\theta}), \quad U^\dagger U = I$$

Z passes both tests. 2Z is Hermitian but fails unitarity.

The diagonal U is unitary for real θ and generally is not Hermitian.

Eigenvalues and eigenvectors

$$Av = \lambda v, \qquad v \neq 0$$

$$(A - \lambda I)v = 0, \qquad \det(A - \lambda I) = 0$$

An eigenvector keeps its direction under the linear map.

Its eigenvalue gives the complex scale factor.

Hermitian eigensolvers and degeneracy

$$AQ = Q\Lambda, \qquad Q^\dagger Q = I, \qquad \Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_n)$$

Columns of Q are orthonormal eigenvectors.

Use numpy.linalg.eigh for a Hermitian matrix.

Repeated eigenvalues define a subspace with many valid bases.

Spectral decomposition

$$A = Q\Lambda Q^\dagger = \sum_{k=1}^{n} \lambda_k \, |q_k\rangle\langle q_k|$$

$$I = \sum_{k=1}^{n} |q_k\rangle\langle q_k|$$

This form applies here to a finite Hermitian matrix.

Each eigenprojector selects one orthonormal direction.

Example 4 · The spectrum of Pauli X

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad |+\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad |-\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

$$X|+\rangle = |+\rangle, \quad X|-\rangle = -|-\rangle, \quad X = P_+ - P_-$$

The eigenvalues are +1 and −1.

The two eigenprojectors sum to the identity.

Functions of a Hermitian matrix

$$A = Q\Lambda Q^\dagger \quad \Longrightarrow \quad f(A) = Q f(\Lambda) Q^\dagger$$

$$f(\Lambda) = \operatorname{diag}\big(f(\lambda_1), \ldots, f(\lambda_n)\big)$$

For polynomials and the exponential, apply f to the eigenvalues.

Elementwise application to the entries of A generally gives a different matrix.

The matrix exponential series

$$e^A = \sum_{k=0}^{\infty} \frac{A^k}{k!} = I + A + \frac{A^2}{2!} + \frac{A^3}{3!} + \cdots$$

The zeroth matrix power is I. Each higher power uses matrix multiplication.

The series converges for every finite square matrix.

Use scipy.linalg.expm for numerical matrix exponentials.

Pauli powers separate even and odd terms

$$X^2 = I, \qquad X^{2k} = I, \qquad X^{2k+1} = X$$

$$e^{-iaX} = \left[\sum_{k=0}^{\infty} \frac{(-1)^k a^{2k}}{(2k)!}\right] I - i \left[\sum_{k=0}^{\infty} \frac{(-1)^k a^{2k+1}}{(2k+1)!}\right] X$$

Even powers contribute an identity term. Odd powers contribute an X term.

Cosine and sine are the two scalar series.

A closed form for an X rotation

$$R_x(\theta) = e^{-i\theta X/2} = \cos\left(\frac{\theta}{2}\right) I - i \sin\left(\frac{\theta}{2}\right) X$$

$$R_x(\pi)|0\rangle = -i|1\rangle, \qquad R_x(0) = I$$

The angle θ is real and measured in radians.

At θ = π, the final state differs from |1⟩ only by a global phase.

Time evolution and the units of the exponent

$$U(t) = e^{-iHt/\hbar}, \qquad H^\dagger = H, \qquad \hbar = 1$$

$$H = \frac{\Omega}{2} X \quad \Longrightarrow \quad U(t) = R_x(\Omega t)$$

This expression assumes a time-independent Hamiltonian.

The course uses time in μs and H/ħ in rad/μs, so Ωt is dimensionless.

Matrix order matters

$$XZ = -ZX, \qquad [X, Z] = XZ - ZX = -2iY$$

$$e^{A+B} = e^A e^B \quad \text{is guaranteed when } AB = BA$$

Arbitrary matrices need not obey the scalar exponential product rule.

Check commutation before splitting an exponential.

Code walkthrough 1 · NumPy and SciPy

```
import numpy as np
from scipy.linalg import expm
X = np.array([[0, 1], [1, 0]], dtype=complex)
values, Q = np.linalg.eigh(X)
theta = 0.7
U = expm(-1j * theta * X / 2)
U_spec = Q @ np.diag(np.exp(-1j*theta*values/2)) @ Q.conj().T
assert np.allclose(U, U_spec)
```

Use @ for matrix products and .conj().T for the adjoint.

Code walkthrough 2 · QuTiP

```
import qutip as qt
Xq = qt.sigmax()
Uq = (-1j * theta * Xq / 2).expm()
zero = qt.basis(2, 0)
final = Uq * zero
p1 = qt.expect(qt.basis(2, 1).proj(), final)
assert np.allclose(Uq.full(), U)
assert np.isclose(p1, np.sin(theta/2)**2)
```

This continues slide 25, with NumPy, θ and U already defined.

Independent numerical checks

$$\epsilon_U = \|U^\dagger U - I\|_{\mathrm{F}}, \qquad \epsilon_{\mathrm{ref}} = \|U - U_{\mathrm{analytic}}\|_{\mathrm{F}}$$

$$r_k = \|Aq_k - \lambda_k q_k\|_2$$

Check structure, a hand-derived reference and an eigenpair residual.

Unitarity alone cannot prove that U implements the intended rotation.

Practice and exit checkpoint

1. Compute the adjoint and normalize the vector from Example 1.

2. Build its projector and test its action on an orthogonal vector.

3. Derive the exponential of an imaginary scalar times Pauli Z.

4. Explain why unitary output alone cannot validate a rotation angle.

Appendix · Notation and dimensions

$$|v\rangle : \ n \times 1, \qquad \langle v| = |v\rangle^\dagger : \ 1 \times n$$

$$\langle u|v\rangle : \ 1 \times 1, \qquad |u\rangle\langle v| : \ n \times n$$

T means transpose. A star means entrywise conjugation here.

A dagger means conjugate transpose. I is the identity operator.

Proof of eigenvector orthogonality

$$Aq_j = \lambda_j q_j, \quad Aq_k = \lambda_k q_k, \quad A^\dagger = A$$

$$\lambda_k \, q_j^\dagger q_k = q_j^\dagger A q_k = \lambda_j \, q_j^\dagger q_k$$

$$(\lambda_k - \lambda_j) \, q_j^\dagger q_k = 0$$

If the real eigenvalues differ, the inner product must be zero.

For a repeated eigenvalue, choose an orthonormal basis within its eigenspace.

Appendix · Practice answers

$$u = \frac{1}{\sqrt{2}}\begin{pmatrix}1\\ i\end{pmatrix}, \qquad P = \frac{1}{2}\begin{pmatrix}1 & -i\\ i & 1\end{pmatrix}$$

$$e^{-iaZ} = \begin{pmatrix}e^{-ia} & 0\\ 0 & e^{ia}\end{pmatrix} = \cos(a)I - i\sin(a)Z$$

The zero vector has no normalized version.

Projector checks: Hermiticity, idempotence and action on its range.

Running on Windows and Ubuntu

```
Windows PowerShell, from the repository root
.\.venv\Scripts\python.exe lectures/S02-linear-algebra/examples.py
.\.venv\Scripts\python.exe lectures/S02-linear-algebra/starter.py

Ubuntu Bash, from the repository root
./.venv/bin/python lectures/S02-linear-algebra/examples.py
./.venv/bin/python lectures/S02-linear-algebra/starter.py
```

Run solution.py separately to check the reference implementation.